



Programmierkurs

Einführung in Java – Tag 4

Inhaltsübersicht



Tag 1	Was ist Java? Variablen, Datentypen, (Konvertierung), Kommentare, Livedemo
Tag 2	Boolsche Ausdrücke, If-Abfragen, (Switch-Case)
Tag 3	Arrays, Schleifen, Debugging
Tag 4	Methoden, Klassen, Objektorientierung

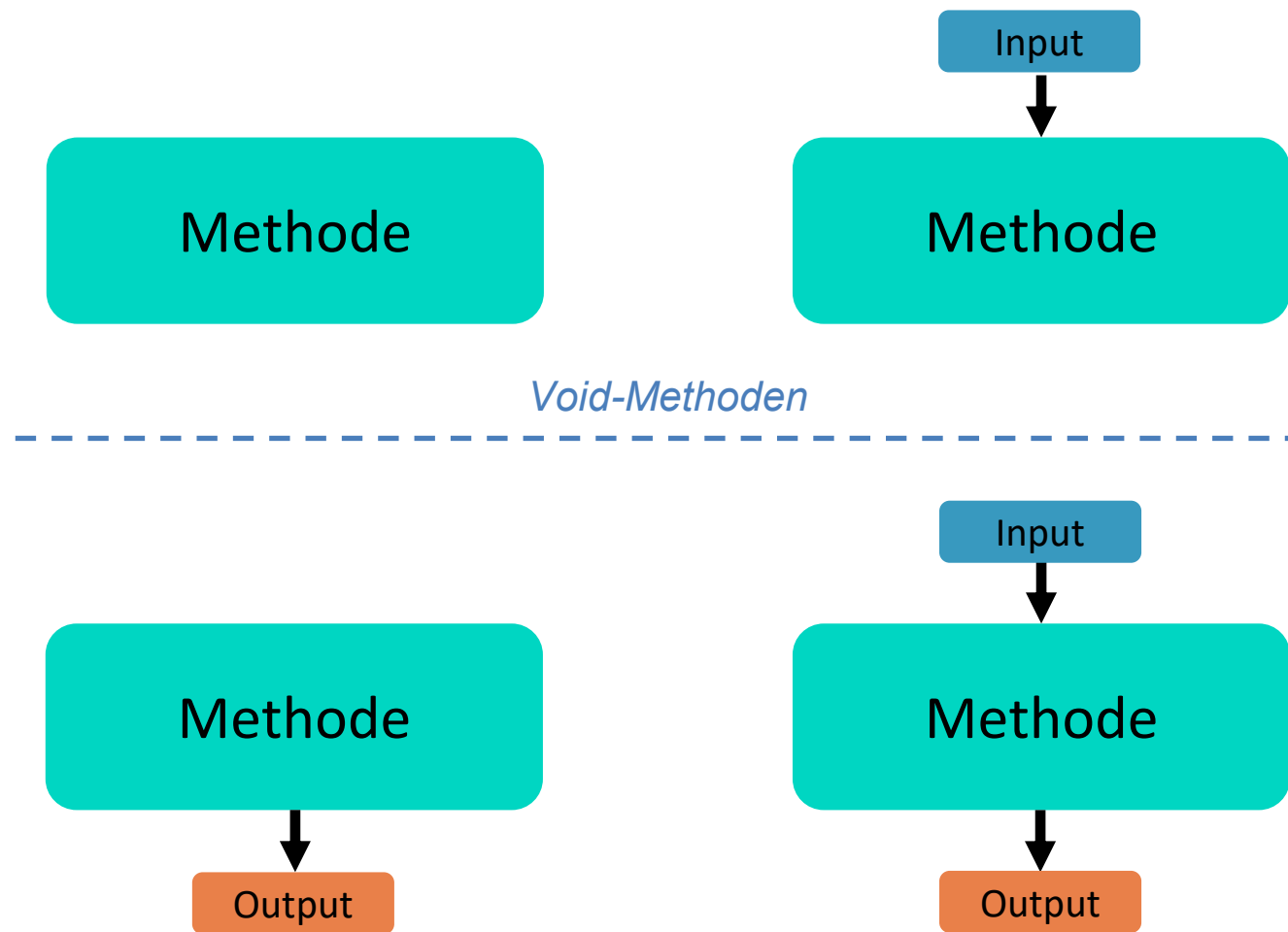
Methoden

- Aufrufbare Codeabschnitte
- Sinnvoll um Code-Duplikation zu vermeiden
- Teilen das Programm in kleine Funktionen auf
- Methoden können Eingaben (Input) annehmen und Ausgaben (Output) zurückgeben

```
1 void doSomething(String text) {  
2     //code...  
3 }
```



Arten von Methoden



Void-Methoden

- Keine Eingabe
- Keine Ausgabe
- Einfachste Form von Methoden

Methode

```
1 public static void main(String[] args) {  
2     doSomething();  
3 }  
4  
5 static void doSomething() {  
6     System.out.println("something!");  
7 }
```

Aufruf der Methode

Deklaration der
Methode

Void-Methoden (mit Eingabe)

- Eingabe // Keine Ausgabe
- Eingabetyp muss in den Klammern der Deklaration festgelegt werden

Deklaration:

Deklaration mit `int`
„Parameter“

```
1 void printIfEven(int number) {  
2     //code...  
3 }
```

Methode erwartet
beim Aufruf eine Zahl

In der Methode wird die
Variable „number“ benutzt
(number = Eingabewert)



Input



Methode

Void-Methoden (mit Eingabe)

- Eingabe // Keine Ausgabe
- Eingabetyp muss in den Klammern der Deklaration festgelegt werden

Aufruf:

Aufruf mit `int` Eingabe

```
1 int value = 5;  
2 printIfEven(value);
```

Methode bekommt
den Wert 5

In der Methode hat
„number“ jetzt den
Wert 5



Input



Methode

Void-Methoden (mit Eingabe)



Input



Methode

Vollständiges Beispiel:

```
1 public static void main(String[] args) {  
2     int value = 5;  
3     printIfEven(value);  
4     value = 42;  
5     printIfEven(value);  
6 }  
7  
8 static void printIfEven(int number) {  
9     if (number % 2 == 0) {  
10        System.out.println(number);  
11    }  
12 }
```

//true, wenn **number** gerade ist

Methoden



- Keine Eingabe // Ausgabe
- Ausgabetyt muss vor dem Namen festgelegt werden
- Methode gibt Wert mit „return“ zurück

Deklaration:

Deklaration mit **double**
als Rückgabetyt

```
1 double getPi() {  
2     return 3.14;  
3 }
```

return gibt den Wert
3,14 zurück

Methode

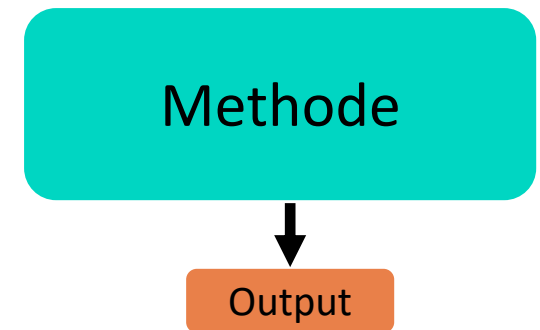


Output

Methoden



- Keine Eingabe // Ausgabe
- Ausgabetyt muss vor dem Namen festgelegt werden
- Methode gibt Wert mit „return“ zurück



Aufruf:

Wird bei Aufruf als Wert behandelt!

```
1 double number = getPi();
```

number hat jetzt den Wert, den getPi() zurück gibt

Methoden (mit Eingabe)

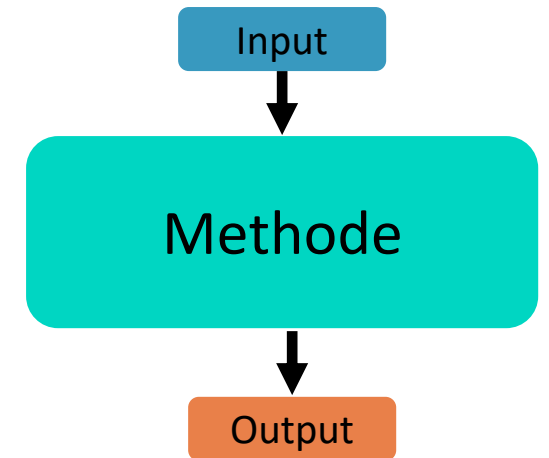
- Eingabe // Ausgabe

Deklaration:

double als Rückgabetyp

float Parameter für Eingabe

```
1 double circleCircumference(float radius) {  
2     double result = 2 * getPi() * radius;  
3     return result;  
4 }
```



Methoden (mit Eingabe)

- Eingabe // Ausgabe

Aufruf:

```
1 float radius = 10.4f;  
2 double circumference = circleCircumference(radius);
```

nimmt float entgegen, gibt double zurück



Input

Methode

Output

Methoden (mehrere Eingaben)

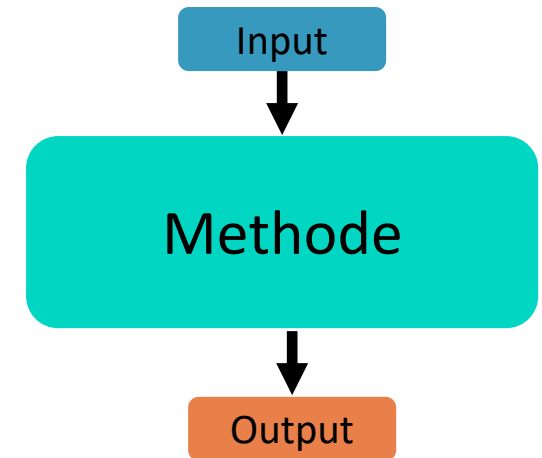


Methode

```
1 int countChars(String text, char symbol) {  
2     int counter = 0;  
3  
4     for (int i = 0; i < text.length(); i++) {  
5         if (text.charAt(i) == symbol) {  
6             counter++;  
7         }  
8     }  
9  
10    return counter;  
11 }
```

Aufruf

```
1 String example = "java";  
2 int amountOfA = countChars(example, 'a'); // = 2  
3 int amountOfV = countChars(example, 'v'); // = 1
```



Main-Methode

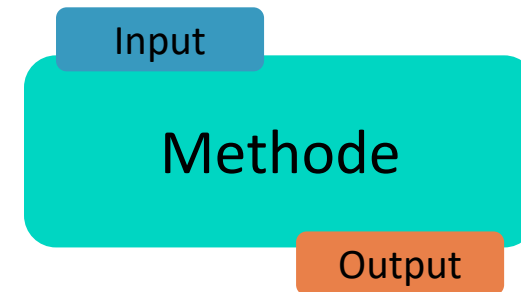
- Methode, die beim Programmstart aufgerufen wird
- Wir haben bisher nur in dieser Methode programmiert

```
1 public static void main(String[] args) {  
2     //code...  
3 }
```

Achtung: Jede Methode, die hier aufgerufen werden soll, muss „**static**“ vor dem Rückgabetypen stehen haben

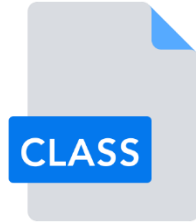
Zusammenfassung

- Aufrufbare Code-Abschnitte
- Beim Aufruf können Daten mitgegeben werden
- Methoden können Daten zurückgeben



→ Wie eine Maschine: erfüllen ihre Funktion und haben Input/Output

Klassen



Bisher hatten wir nur Datentypen für:

- Zahlen (int, float, double)
- Zeichen (char, String)
- Wahrheitswert (boolean)

Was, wenn wir jetzt z.B. eine Person speichern wollen?

Klassen - Was ist das?



Bisher hatten wir nur Datentypen für:

- Zahlen (int, float, double)
- Zeichen (char, String)
- Wahrheitswert (boolean)

Was, wenn wir jetzt z.B. eine Person speichern wollen?

Eine Person hat z.B. die Daten:

- Name (String)
- Alter (int)
- Größe (float)

Klassen - Was ist das?

Eine Person hat z.B. die Daten:

- Name (String)
 - Alter (int)
 - Größe (float)
- 
- Person

Idee von Klassen:

- Diese Daten gebündelt in einem Paket
- Dieses Paket „Person“ als Variable benutzbar machen

Klassen - Allgemein

- „**class**“ definiert eine neue Klasse
- Jede Klasse hat eigene Datei
- Klassen haben eigene Variablen (Klassenvariablen) & Methoden
- Methoden können auf Klassenvariablen zugreifen
- Wir haben bisher in einer Methode (main) einer Klasse programmiert

```
1 public class ExampleClass {  
2  
3     int amount = 6;  
4  
5     void increase() {  
6         amount++;  
7     }  
8 }
```

increase() greift hier
auf *Klassenvariable*
amount zu

Klassen

Erstellen:

- Neue Datei mit „class“ und Klassenname
- Daten als Klassenvariablen festlegen („Attribute“)

z.B. Klasse Person:

```
1 class Person {  
2  
3     String name;  
4     int age;  
5     float height;  
6     boolean isHappy = true;  
7 }
```

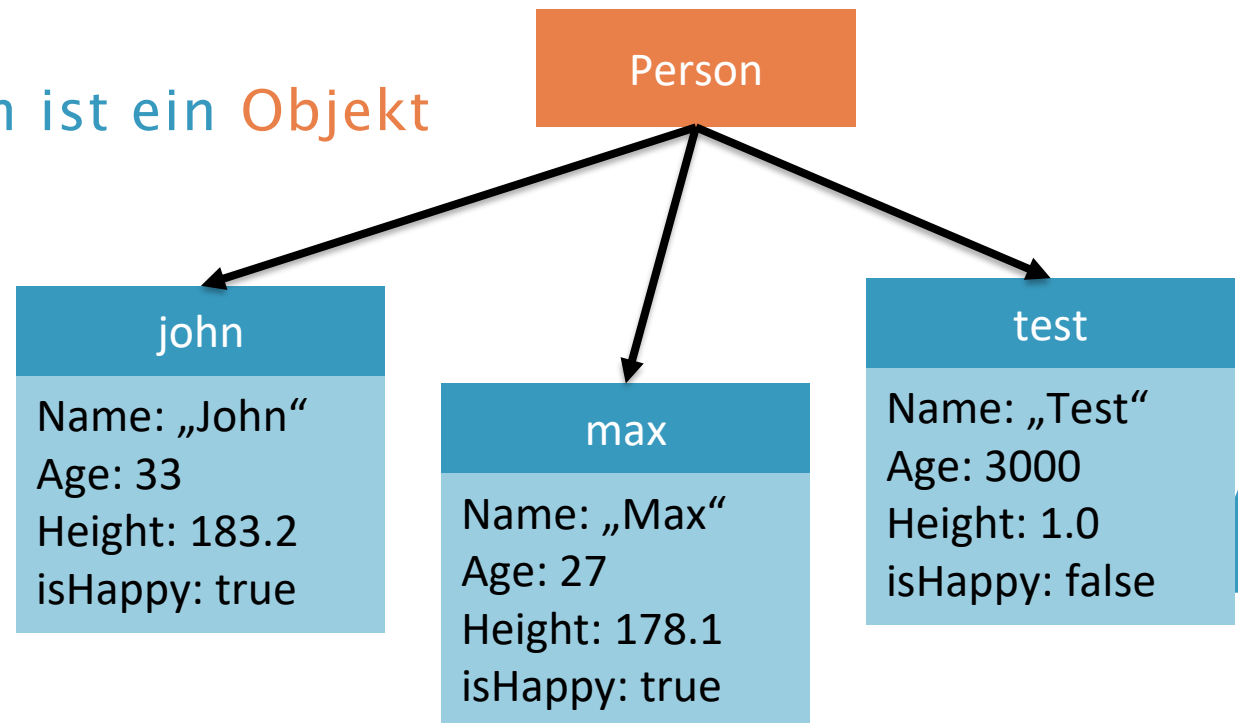
Klasse „Person“
enthält hier alle für
uns relevanten Daten
zu einer Person

(alle „Attribute“
einer Person)

Klassen - Objekte

- Ein „Objekt“ ist eine Instanz einer Klasse
- Werden wie Variablen der Klasse behandelt

z.B. eine spezifische Person ist ein **Objekt** der Klasse **Person**



Klassen - Objekte

Variable (Objekt) der Klasse erzeugen:

```
1 Person max = new Person();
```

Datentyp

Variablenname

„neues Objekt“

...von Klasse „Person“

Klassen - Objekte

- Daten von Objekt werden mit Punkt aufgerufen
- Daten können zugewiesen und gelesen werden

```
1 Person max = new Person();  
2  
3 max.name = "Max";  
4 max.age = 27;  
5 max.height = 178.1f;  
6  
7 String text = max.name;  
8  
9 System.out.println(max.age);
```

Klassen und Methoden

- Objekte haben neben Attributen auch Methoden
- Fügt dem Objekt Funktion hinzu (ermöglicht es „etwas zu tun“)

```
1 class Dog {  
2     String name;  
3  
4     void bark() {  
5         System.out.println("wuff!");  
6     }  
7 }
```


Klassen und Methoden

- Methoden eines Objekts werden auch mit Punkt aufgerufen
- Methoden werden normal behandelt (können input/output/nichts von beidem haben)

```
1 Dog corgi = new Dog();  
2 corgi.name = "hund"; //TODO: need better name  
3  
4 corgi.bark();
```

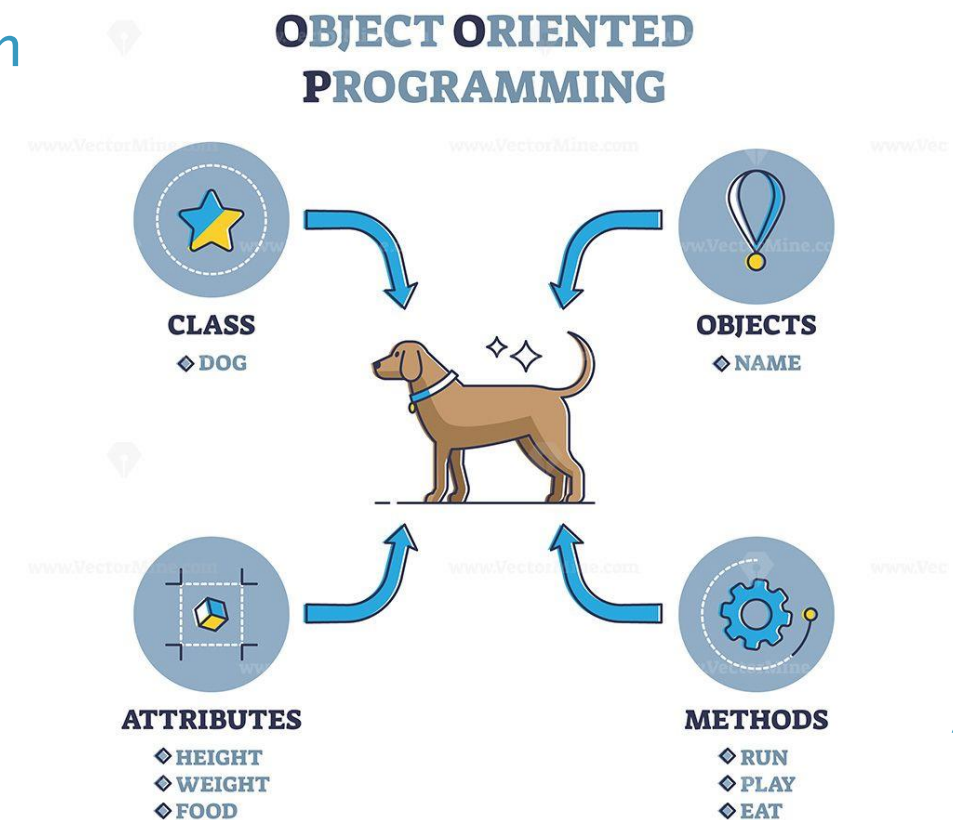
```
wuff!
```

```
Process finished with exit code 0
```

Objektorientierung (OOP)

- Modellierung der Klassen nach der Wirklichkeit
- Bestimmt den Aufbau der meisten Programme in Java
- Fundamentaler Programmierstil

(OOP umfasst noch mehr Themen, die im Vorkurs erstmal weggelassen werden)



Objektorientierung

Modellierung eines Programms nach der Wirklichkeit:

- **Klassen** = Bauplan für Objekt
- **Objekt** = Objekt
- **Attribute** = Eigenschaften
- **Methoden** = Fähigkeiten/Funktionen

Sollte nur die **relevanten** Eigenschaften/Funktionen abdecken!



Klasse

Apfel

color
size

eatApple()

Wirklichkeit

Eigenschaften

- Farbe
- Größe

Funktion

- essen

OOP – Beispiel

